

ZADÁNÍ PRAKTICKÉHO ÚKOLU

Software Test Engineer (Hybrid E2E Automation & CI/CD)

Název úkolu: Hybridní E2E automatizace (API + UI) s integrací do CI/CD

Předpokládaný čas: 4–6 hodin

Cíl úkolu: Navrhnout a implementovat robustní, stabilní a snadno udržovatelný automatizační framework pro komplexní scénář kombinující backendové API a frontendové UI.

1. Technické požadavky na řešení

- **Jazyk a framework:** Playwright (TypeScript nebo C# v závislosti na tvých preferencích).
- **Architektura:** Použij Page Object Model (POM) nebo jiný uznávaný architektonický návrhový vzor vhodný pro testování.
- **Verzování:** Kód nahraj do veřejného (nebo s námi nasdíleného soukromého) repozitáře na GitHubu.
- **CI/CD:** Přidej integraci (např. GitHub Actions), která automaticky spustí testy při pushnutí kódu.

2. Testovací scénář (UI E2E Workflow)

Cílem je zautomatizovat ucelený nákupní proces na testovací aplikaci **Demoblaze** (<https://www.demoblaze.com/>):

1. **Otevření webu:** Přejdi na domovskou stránku a ověř její úspěšné vykreslení a správnost URL.
2. **Výběr produktu:** Dynamicky vyber konkrétní produkt z mřížky (např. 'Samsung galaxy s6' nebo 'Nokia lumia 1520') a klikni na něj.
3. **Detail produktu:** Ověř, že se správně načetla stránka s detailem vybraného produktu (ověř URL a shodu názvu produktu v nadpisu).
4. **Přidání do košíku:** Klikni na tlačítko 'Add to cart' a asynchronně se vypořádej s potvrzovacím dialogem (alertem) prohlížeče.
5. **Přechod do košíku:** Přejdi do košíku pomocí hlavní navigační lišty.
6. **Ověření košíku:** Ověř, že se stránka košíku úspěšně načetla a že tabulka položek obsahuje přesně ten produkt, který jsi přidal/a.

Architektonické požadavky na UI část:

- **Page Objects:** Rozděl kód na samostatné stránky (HomePage, ProductDetailPage, CartPage a sdílený NavigationHeader).
- **Page Guards:** Každý objekt by měl v metodě `isLoading()` ověřit URL a přítomnost klíčových komponent.
- **Žádné statické sleep:** Používej výhradně dynamická a asynchronní čekání (např. čekání na dialogový event).

3. Testovací scénář (API Integration Workflow)

Pro tento úkol použij veřejné testovací API **Restful-Booker**:

- **Základní URL:** <https://restful-booker.herokuapp.com>
- **Dokumentace API:** <https://restful-booker.herokuapp.com/apidoc/index.html>

Tvým úkolem je naautomatizovat ucelený integrační scénář (tzv. 'Happy Path' a následný cleanup):

1. Vytvoření rezervace (POST /booking):

- Pošli požadavek na vytvoření nové rezervace s libovolnými testovacími daty (např. viz příklad níže).
- Ověř, že server vrátil status kód 200 OK.
- Ověř, že odpověď obsahuje vygenerované *bookingid* a uložená data odpovídají odeslaným.
- ID vytvořené rezervace si asynchronně ulož pro další krok.

2. Čtení a ověření dat (GET /booking/{id}):

- Pomocí uloženého *bookingid* se dotaž na detail této konkrétní rezervace.
- Ověř, že server vrátil status kód 200 OK.
- Ověř, že vrácený JSON objekt obsahuje správná a nezměněná data z kroku 1.

// Příklad testovacích dat (bookingData):

```
{
  "firstname": "Milena",
  "lastname": "Stranska",
  "totalprice": 1500,
  "depositpaid": true,
  "bookingdates": {
    "checkin": "2026-08-14",
    "checkout": "2026-08-19"
  },
  "additionalneeds": "Breakfast"
}
```

Architektonické požadavky na API část:

- Použij **Playwright API Request Context** (vestavěný HTTP klient).
- V případě TypeScriptu definuj **typová rozhraní (Interfaces)** pro request payload i response body pro zajištění typové bezpečnosti.

4. SQL & Cleanup (Teoretická / Přípravná část)

Představ si, že tato aplikace běží nad PostgreSQL databází a že tvůj e-shop má v databázi dvě propojené tabulky:

1. **customers** - obsahuje sloupce: *id, firstname, lastname, email*
2. **bookings** - obsahuje sloupce: *id, customer_id, total_price, deposit_paid*

Obě tabulky jsou propojené přes ID zákazníka (cizí klíč *customer_id*).

Do souboru **README.md** v repozitáři napiš krátké, čisté SQL dotazy pro následující operace:

- **Úklid dat (DELETE):** SQL dotaz, který po skončení testů bezpečně vyčistí/smaže testovací data z tabulek *bookings* a *customers* pro vytvořeného uživatele (s ohledem na integritu databázových vazeb a cizích klíčů).
- **Ověření uložení adresy (JOIN):** SQL dotaz, který by ověřil (spojil tabulky *users* a *addresses*), zda se nově registrovanému uživateli správně uložila jeho adresa.

5. CI/CD Pipeline & GitHub Actions

Navrhni a vytvoř konfigurační YAML soubor pro GitHub Actions, který zajistí automatické spouštění:

1. **headless spouštění:** Testy se budou spouštět v bezhlavém režimu (např. Chromium) při každém pushnutí kódu do hlavní větve.
2. **Ukládání reportů (Artifacts):** V případě jakéhokoli selhání testu vygeneruj a ulož test report (HTML, screenshoty nebo video) jako artefakt s životností 30 dní.
3. **Separace:** (Volitelně) API a UI testy by měly v pipeline běžet v oddělených jobech pro maximální efektivitu a rychlost.

6. Co očekáváme k odevzdání

Při technické prezentaci budeme hodnotit komplexní inženýrský přístup, kvalitu kódu a architekturu řešení. K odevzdání prosím připrav:

1. **Odkaz na GitHub repozitář** s přehlednou strukturou kódu.
2. **README.md v angličtině** obsahující:
 - Návod pro instalaci závislostí a lokální spuštění testů (např. *npm install && npm run test*).
 - SQL dotazy z Části 4.
 - Krátké odůvodnění tvé volby architektury a způsobu asynchronního čekání.
3. **Technická prezentace (15–20 minut):** Během schůzky nám ukážeš kód, spustíš testy lokálně, předvedeš výsledek v GitHub Actions a společně prodiskutujeme tvé řešení.

Poznámka ke kvalitě kódu:

Celý kód, názvy testů, lokátory i dokumentace by měly být napsány v **angličtině**. Klíčové metody v Page Objectech doplň o JSDoc komentáře. Hledáme čistý, čitelný a dlouhodobě udržitelný kód.